

# AI-BDM

What it does, how it works, and how to try it yourself — in plain language.

---

This document explains the AI-BDM project without assuming you already know how it works — what it does, why each part exists, and exact copy-and-paste steps to run it and see real results. If you're testing this project, read this first; the detailed test-case spreadsheet (AI-BDM\_QA\_Test\_Plan.xlsx) is the companion document for the actual testing work.

# 1. What Is AI-BDM, In One Paragraph?

Imagine you needed to find **3 dental clinics in Islamabad that don't currently let patients book appointments online**. Doing that by hand means: searching Google, opening a dozen websites, reading each one to check if they have online booking, checking if they're even a real, currently-operating business, and writing it all down. AI-BDM does all of that automatically. You type one plain-English sentence, and it hands you back a list of real businesses that actually match — with the evidence for every claim, not just a guess.

It's built for B2B lead generation — finding real companies that match a very specific description, the kind of research a sales or marketing team would otherwise do by hand.

## What you put in, and what you get back

**You type:**

"find 3 dental clinics in Islamabad with no online booking"

**You get back:**

A list of real, currently-operating dental clinics in Islamabad, each with its name, website, contact details, a confirmed match against Google Maps, and a plain answer to the actual question asked ("does this business offer online booking? here's why we think so") — grounded in the real text found on their website, never a guess.

## 2. How It Works, Step By Step

Behind the scenes, one request goes through 8 steps, in order. Each step has a plain-English name here and its real technical name in the code, in parentheses, so you can connect the two when reading the deeper documentation later.

1

### **Understand the request** (Query Planning)

Reads your one sentence and figures out the real, structured meaning behind it: what industry, what location, how many results you want, and whether there's a specific extra question to answer (like "...with no online booking"). If your request is too vague to act on, it stops here and asks you a clarifying question instead of guessing.

2

### **Search and read real websites** (Discovery & Scraping)

Searches the web for real candidate businesses, then actually visits each one's website and reads the page content — the same way a person would open a browser tab and read the page. Businesses that clearly aren't a real match are filtered out here.

3

### **Pick the right pages to focus on** (Route/Page Selection)

A business's website might have 20 pages. This step figures out which few pages actually matter for answering your question, instead of re-reading everything every time.

4

### **Answer your specific question** (Final Reasoning)

Only runs when your request had an extra condition (like "...with no online booking"). Reads the relevant page text and gives a direct answer, always backed by the actual text it read — if the evidence isn't there, it says so honestly instead of making something up.

5

### **Identify their website's technology** (Tech-Stack Detection)

Figures out what software a business's website runs on — what platform it's built with, what booking/CRM/marketing tools it uses. Useful when the request is about technology ("...using WordPress").

6

### **Build a searchable memory of everything found** (Evidence Index)

Organizes everything read in step 2 (website text and customer reviews) into a searchable index, so later steps can quickly find the exact right piece of evidence instead of re-reading everything from scratch.

7

### **Double-check against Google Maps** (Maps Enrichment)

Cross-checks each business against its real Google Maps listing — confirming its address, phone number, and rating actually match, so you're not looking at a wrong or outdated business.

8

### **Final quality check** (Accuracy Audit)

Before handing anything back, it re-checks its own work: are phone numbers/emails formatted correctly, does the business name actually match across every source, do the reviews it found genuinely mention this business? Anything questionable gets flagged, not hidden.

Steps 4, 5, 6, and 7 only run when your request actually needs them — a simple "find 3 dental clinics in Islamabad" with no extra condition skips straight from step 2 to step 8.

## 3. Try It Yourself — Complete Steps

Everything below is copy-and-paste ready. Run these from a terminal inside the project folder.

### Step 1 — Install Python 3.12

If you don't already have it, download and install Python 3.12 from [python.org](https://python.org). This project is built and tested specifically on 3.12.

### Step 2 — Create an isolated environment for this project

This keeps AI-BDM's dependencies separate from anything else on your machine. Windows PowerShell:

```
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
```

Mac/Linux:

```
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
```

### Step 3 — Install the project's dependencies

```
pip install -r requirements.txt
```

This installs every package the project needs. It can take a few minutes the first time. Afterward, run this one cleanup command (safe, explained in README.md):

```
pip uninstall -y pyarrow datasets
```

### Step 4 — Get two free API keys

AI-BDM needs two outside services to actually work: one that understands language (Groq) and one that searches the web (Serper.dev). Both have a free tier, enough to try this out.

- Groq — sign up at [console.groq.com](https://console.groq.com) and create an API key. This is what lets the program understand your sentence and reason about what it reads.
- Serper.dev — sign up at [serper.dev](https://serper.dev) and create an API key. This is what lets the program search Google and look things up on Google Maps.

### Step 5 — Add your keys to the project

- 1 In the project folder, find the file named `.env.example`.
- 2 Make a copy of it named exactly `.env` (no extension after that).
- 3 Open `.env` in a text editor and paste your Groq key after `groq_llm_apikey1=` and your Serper key after `serper=`.
- 4 Save the file. Never share this file or commit it anywhere — it holds your real keys.

### Step 6 — Run your first real query

From the project folder, with your environment activated:

```
python main.py --query "find 3 dental clinics in Islamabad with no online booking"
```

## What you'll see happen

- 1 The program prints what it understood your request to mean (its "plan").
- 2 It starts searching and shows each candidate business it finds and checks.
- 3 For each real match, it shows what it scraped, whether reviews were found, and (if relevant) the direct answer to your extra question.
- 4 It cross-checks against Google Maps.
- 5 It runs its own final quality check.
- 6 It prints a summary: how many businesses it found, how many genuinely qualified, and where the full results were saved.

A simple 3-business query typically finishes in a few minutes — most of that time is the program actually waiting on real websites and APIs to respond, the same as if a person were doing this by hand, just faster and more thorough.

## Step 7 — Where to find the results afterward

|                            |                                                                                                                |
|----------------------------|----------------------------------------------------------------------------------------------------------------|
| <b>leads_clean.csv</b>     | One row per business found — the main results file. Open it in Excel.                                          |
| <b>leads_with_maps.csv</b> | The same list, with Google Maps details added (rating, review count, confirmed match).                         |
| <b>accuracy_report.txt</b> | The program's own self-check — anything it flagged as questionable.                                            |
| <b>storage/</b> folder     | One folder per business, with the actual page text and reviews it read — the raw evidence behind every result. |

## 4. Optional: Trying the Web API Version

Everything above runs directly from a terminal. There's also a web version (an API) that other programs can call over the network — useful for understanding how a real backend system would talk to AI-BDM. This part covers only the two most basic requests; the QA Test Plan spreadsheet has the complete set.

### Step 1 — Start the server

```
python -m uvicorn api:app --reload --port 8000
```

### Step 2 — Open the interactive docs in your browser

Go to <http://127.0.0.1:8000/docs> in any browser. This page lets you try every request by clicking buttons, no coding needed — a good way to explore without memorizing commands.

### Step 3 — A simple check that it's alive

```
curl http://127.0.0.1:8000/api/v1/health
```

Should return a short message confirming the service is running and which of your API keys it detected.

## 5. Where To Go Next

- **AI-BDM\_QA\_Test\_Plan.xlsx** — the full, detailed list of 142 test cases covering every part of the project. Use this once you're comfortable with what the project does and want to start structured testing.
- **README.md** (in the project folder) — the technical setup reference, if anything above doesn't match what you see.
- **docs/** folder — deeper technical documentation for every part of the system, for when a test case raises a question this document doesn't answer.